Approximate Linear Programming for Decentralized Policy Iteration in Cooperative Multi-agent Markov Decision Processes

Lakshmi Mandal[1], Chandrashekar Lakshminarayanan[2], and Shalabh Bhatnagar[1]

arXiv:2311.11789v2 [cs.LG] 29 Apr 2024

Abstract

In this work, we consider a 'cooperative' multi-agent Markov decision process (MDP) involving $m(> 1)$ agents. At each decision epoch, all the m agents independently select actions in order to maximize a common long-term objective. In the policy iteration process of multi-agent setup, the number of actions grows exponentially with the number of agents, incurring huge computational costs. Thus, recent works consider decentralized policy improvement, where each agent improves its decisions unilaterally, assuming that the decisions of the other agents are fixed. However, exact value functions are considered in the literature, which is computationally expensive for a large number of agents with high dimensional state-action space. Thus, we propose approximate decentralized policy iteration algorithms, using approximate linear programming with function approximation to compute the approximate value function for decentralized policy improvement. Further, we consider (both) cooperative multi-agent finite and infinite horizon discounted MDPs and propose suitable algorithms in each case. Moreover, we provide theoretical guarantees for our algorithms and also demonstrate their advantages over existing state-of-the-art algorithms in the literature.

I. Introduction

Markov Decision Process (MDP) provides a mathematical framework for modeling problems involving sequential decision-making under uncertainty arising in fields such as artificial intelligence, finance, management, manufacturing and adaptive control [1]. Here, the goal is to choose actions in response to the states of the system so as to maximize a certain long-term objective. Based on the number of decision-makers (also known as agents) being one or more, MDPs can be classified as 'single-agent' MDP (SA-MDP) or 'multi-agent' MDP (MA-MDP). Based on the objective of the individual agents, MA-MDP can be further classified as 'competitive' or 'cooperative'. In competitive MA-MDP, the m agents ($m > 1$) in general have competing objectives that can often be in conflict with those of other agents [2], [3]. In cooperative MA-MDP, all agents cooperate in order to maximize a common objective function [4], [5].

Dynamic programming (DP) methods, namely value iteration, policy iteration, and linear programming, are three basic numerical solution methods for MDPs [6], [7]. These are exact and full-state methods to find the optimal value and policy functions. However, MDPs arising in practice have a large number of states, and by the curse-of-dimensionality (COD), the number of states grows exponentially in the number of state variables. Thus, DP methods are impractical in the setting of MA-MDP with high-dimensional state space due to the excessive computational costs. Approximate dynamic programming (ADP) methods [8], [9] overcome COD by approximating value function via a parameterized family of functions and finding the optimal parameter whose dimension is much smaller in comparison to the number of states.

Policy improvement is a key step in both exact as well as approximate DP methods and maps a candidate value function to its corresponding 'one-step' greedy policy. In the case of MA-MDP, an important issue (in addition to COD) is that the overall action is a Cartesian product of the actions of the individual agents and hence the number of actions also grows exponentially in the number of

[1]Department of Computer Science and Automation, Indian Institute of Science, Bangalore 560012. E-mail: {lmandal,shalabh}@iisc.ac.in

[2]Department of Computer Science and Engineering, Indian Institute of Technology Madras, Chennai 600036. E-mail: chandrashekar@cse.iitm.ac.in

*S.Bhatnagar was supported in this work through a J. C. Bose Fellowship of SERB, Project No. DFTM/ 02/ 3125/M/04/AIR-04 from DRDO under DIA-RCOE and the Walmart Centre for Tech Excellence, IISc.

agents [10]. Thus even if the number of states is small, policy improvement could be a computationally expensive operation due to the large number of actions. To address this issue, decentralized policy improvement is proposed in recent literature [11], [12][Chap. 5]. In these works, the exact value function is centrally calculated, while policy improvement is decentralized, whereby each agent independently improves its own policy, assuming policy of the other agents is fixed. However, in the MA-MDP settings, computing the exact value function centrally is computationally expensive. In [13] fully decentralized multi-agent algorithm, with function approximation, on infinite horizon discounted MDP is considered. In the SA-MDP setting, approximate linear programming (ALP) approaches are well studied in the literature [14]. However, in MA-MDP, the ALP-based approach to approximate decentralized policy iteration is not considered in the literature.

Thus, we propose an ALP-based method for decentralized policy iteration (called approximate decentralized policy iteration (ADPI)) in the cooperative MA-MDP setting. The following are the broad contributions of our work:

(1) We propose approximate decentralized policy iteration (ADPI) algorithms for MA-MDPs that can work under large state-action settings under (i) finite horizon and (ii) infinite horizon MDPs. Further, unlike [11], [12], our proposed algorithms compute the approximate value function, resulting in computational savings.

(2) We provide a detailed analysis and theoretical guarantees for the proposed ADPI algorithms.

(3) We empirically demonstrate the effectiveness of our proposed algorithms on standard multi-agent tasks where even with more number of agents and complex state-action spaces, our algorithms outperform [11], [13].

The rest of the paper is organized as follows. Section II describes the Background. Section III presents the proposed algorithms as well as the theoretical analysis. The experimental setting and numerical results are provided in section IV. Finally, Section V provides the concluding remarks.

II. Background

A. Single Agent Markov Decision Process

Markov Decision Process (MDP) is a mathematical framework to cast problems involving sequential decision making under uncertainty. Formally, an MDP is given by the 4-tuple $\langle \mathcal{S}, \mathcal{U}, g, P \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{U}$ is the action space, g is the single-stage cost and P denotes the transition probability kernel. In this paper, we consider MDP with finite state-action space. Here, for $x, y \in \mathcal{S}$ and $u \in \mathcal{U}(x)$, $p_{xy}(u)$ is the probability of transitioning from state x to y while choosing action u and $g_{xy}(u)$ denotes the associated cost.

Policy and Value: The behaviour of the agent is formally described by a 'policy' denoted by $\pi = \{\mu_0, \mu_1, \dots, \mu_k, \dots\}$, where, $\mu_k : \mathcal{S} \to \mathcal{U}$, $k = 0, 1, \dots$, are mappings from the state space $\mathcal{S}$ to the action space $\mathcal{U}$. The value function $J_\pi \colon \mathcal{S} \to \mathbb{R}$ captures the long-term cumulative objective obtained by the agent acting according to a policy π starting from each state. In this paper, we consider two kinds of long-term objectives: (i) finite horizon total cost (FHTC) and (ii) infinite horizon discounted cost (IHDC), whose value functions are as follows.

$$\text{(FHTC)} : J_\pi(x) = \mathbb{E}[\sum_{k=0}^{N} g_{x_k x_{k+1}}(\mu_k(x_k)) | x_0 = x], \tag{1}$$

$$\text{(IHDC)} : J_\pi(x) = \mathbb{E}[\sum_{k=0}^{\infty} \alpha^k g_{x_k x_{k+1}}(\mu_k(x_k)) | x_0 = x], \tag{2}$$

where $\alpha \in (0, 1)$ is the discount factor. In what follows, in this section, we briefly discuss the basic ideas using the IHDC setting (those related to the FHTC setting are similar and can be found in [1]).

Policy Improvement (PI): Given an MDP, we are interested in finding the optimal policy π_* and its associated optimal value function J_*. The 'one-step' greedy operator also known as the *Bellman*

Operator plays a central role in the solution methods for obtaining the optimal quantities. In the IHDC case, for a given $J\colon \mathcal{S} \to \mathbb{R}$, the Bellman operator and its associated 'one-step' policy improvement operator are, respectively, as follows.

$$(\mathcal{T}J)(x) = \min_{u \in \mathcal{U}(x)} \sum_{y} p_{xy}(u) \left[g_{xy}(u) + \alpha J(y) \right], \tag{3}$$

$$(\mathcal{G}J)(x) = \arg\min_{u \in \mathcal{U}(x)} \sum_{y} p_{xy}(u) \left[g_{xy}(u) + \alpha J(y) \right]. \tag{4}$$

Further, for the IHDC case, for a given policy μ, $J\colon \mathcal{S} \to \mathbb{R}$, the Bellman operator $\mathcal{T}_\mu$ is defined as follows:

$$(\mathcal{T}_\mu J)(x) = \sum_{y} p_{xy}(\mu(x)) \left[g_{xy}(\mu(x)) + \alpha J(y) \right]. \tag{5}$$

Dynamic Programming (DP) Approaches: The basic numerical solution approaches to calculate π_* and J_* are value iteration, policy iteration and linear programming. The first two are well-studied approaches, and most RL algorithms are based on one of these. Linear programming casts the Bellman equation $J_* = \mathcal{T}J*$ as

$$\begin{aligned} J_* &= \arg\max_{J \in \mathbb{R}^{|\mathcal{S}|}} c^\top J \\ s.t \quad & J(x) \leq \sum_{y} p_{xy}(u) \left[g_{xy}(u) + \alpha J(y) \right], \forall x \in \mathcal{S}, u \in \mathcal{U}(x), \end{aligned}$$

and obtains the optimal policy as $\pi_* = \mathcal{G}J_*$.

Approximate Dynamic Programming (ADP) Methods: Most MDPs arising in practical applications have a large number of states. Hence, it is computationally expensive to calculate J_* and π_*, and DP-based approaches cannot be directly used due to the COD problem. ADP methods handle the issue of large number of states by introducing function approximation, i.e., they approximate the value function $J_\pi \approx \Phi r$, where $\Phi \in \mathbb{R}^{|\mathcal{S}| \times d}$ is a *feature matrix* and $r \in \mathbb{R}^d$ is a parameter (with $d << |\mathcal{S}|$). Approximate value iteration, approximate policy iteration, and approximate linear programming are some of the widely studied ADP methods.

B. Multi-Agent Markov Decision Process

In this paper, we consider the 'cooperative' multi-agent MDP (CO-MA-MDP) with 'm' agents, where at each stage, the action is given by $u = (u^1, \ldots, u^m)$, with u^i being the action of agent i. Hence, the overall action space is given by $\mathcal{U} = \mathcal{U}^1 \times \mathcal{U}^2 \times \cdots \times \mathcal{U}^m$, where $\mathcal{U}^i$ is the action space of agent i and, $u^i \in \mathcal{U}^i(x), x \in S$. As a result, the size of the action space grows exponentially in the number of agents. To see its impact, note that while the underlying equations are still (3)-(5) in the case of the CO-MA-MDP, in the minimization in these equations, the action u now has m components each corresponding to an agent as opposed to a single component in equations (3) and (4). Further, in (5), the policy μ in the MA case has m components $\mu = (\mu^1, \ldots, \mu^m)$, with each component corresponding to an agent as opposed to (5).

C. Prior Work: Decentralized Policy Iteration

Due to the fact that the action $u = (u^1, \ldots, u^m)$ has m components, it becomes expensive to exactly compute the minimization in equation (3)-(4). In order to address this issue, [11] proposed an 'agent-by-agent' decentralized policy improvement (DPIm), wherein, each agent improves its own policy component, assuming that the other agents continue working with their policy components fixed. In particular, when the ith agent improves its policy, agents $1, 2, \ldots, i-1$ would have already updated their policies in the current cycle and would be choosing actions according to their updated policies $\tilde{\mu}^1, \tilde{\mu}^2, \ldots, \tilde{\mu}^{i-1}$, respectively, whereas agents $i+1, i+2, \ldots, m$ would not have updated their policies until then and

would continue to pick actions as per the *base* policy namely $\mu^{i+1}, \mu^{i+2}, \ldots, \mu^m$ respectively. The DPIm in [11] is shown in Algorithm 1, where $\tilde{\mu}_{1:i-1} = (\tilde{\mu}^1, \ldots, \tilde{\mu}^{i-1})$ denotes collectively the updated policies of agents $1, \ldots, i-1$ and $\mu_{i+1:m} = (\mu^{i+1}, \ldots, \mu^m)$ denotes collectively the *base* policy of agents $i+1, \ldots, m$. As discussed in [11], the DPIm in Algorithm 1 requires a form of *weak* communication between agents, wherein agent i can obtain the knowledge of $\tilde{\mu}^j, j = 1, \ldots, i-1$ (to achieve this, it is enough if agent $i-1$ can communicate with agent i). However, in the literature, DPIm with exact value function is considered, which is computationally expensive with a large number of agents and/or with large state-action space [9].

Algorithm 1 Decentralized Policy Improvement (DPIm):
DPIm$(i, \tilde{\mu}_{1:i-1}, \mu_{i+1:m}, J, \alpha)$

Input: agent i; updated policies $\tilde{\mu}_{1:i-1}$; base policies $\mu_{i+1:m}$; value function J
$\widetilde{\mu}^i(x) \in \arg\min_{u^i \in \mathcal{U}^i(x)} \sum_{y \in S} p_{xy}(\widetilde{\mu}^1(x), \ldots, \widetilde{\mu}^{i-1}(x),$
$u^i, \mu^{i+1}(x), \ldots, \mu^m(x))[g_{xy}(\widetilde{\mu}^1(x), \ldots, u^i, \ldots, \mu^m(x))$
$+\alpha J(y)], \forall x \in S$
Return updated policy $\widetilde{\mu}^i$ for the agent i.

Algorithm 2 Calculate approximate cost function: CACFN$(k, \widetilde{\pi}, J^{\text{ALP}}_{k+1,\widetilde{\pi}})$

Input: $k, \widetilde{\pi}, J^{\text{ALP}}_{k+1,\widetilde{\pi}}$
$J^{\text{ALP}}_{k,\widetilde{\pi}} = \max_{r \in \mathbb{R}^d} c^\top \Phi r,\ c \in \mathbb{R}^n, \Phi \in \mathbb{R}^{n \times d}$
Subject to: $\Phi r(x) \leq \sum_{y \in S} p_{xy}(\widetilde{\mu}^1_k(x), \ldots, \widetilde{\mu}^m_k(x))$
$[g_{xy}(\widetilde{\mu}^1_k(x), \ldots, \widetilde{\mu}^m_k(x)) + J^{\text{ALP}}_{k+1,\widetilde{\pi}}(y)], \forall x \in S$
Return $J^{\text{ALP}}_{k,\widetilde{\pi}}$

Algorithm 3 DPI using ALP for FH CO-MA-MDP

Input: N, n, m
$u_k = (u^1_k, u^2_k, \ldots, u^m_k) \in \mathcal{U}^1_k(x) \times \mathcal{U}^2_k(x) \times \cdots \times \mathcal{U}^m_k(x)$, for state $x \in S$.
$\pi = \{\mu_0, \mu_1, \ldots, \mu_k, \ldots, \mu_{N-1}\}$, where $\mu_k = (\mu^1_k, \mu^2_k, \ldots, \mu^m_k)$.
Call CACFN$(k, \pi, J^{\text{ALP}}_{k+1,\pi})$, i.e., Algorithm 2 to get $J^{\text{ALP}}_{k,\pi}$.
Terminal cost $J^{\text{ALP}}_{N,\widetilde{\pi}} = J^{\text{ALP}}_{N,\pi} = g_N$.
for stages $k = N-1, N-2, \ldots, 0$ **do**
  repeat
    for agent $i = 1, 2, \ldots, m$ **do**
      Call Algorithm 1 to get $\widetilde{\mu}^i_k$.
    end for
    Call CACFN$(k, \widetilde{\pi}, J^{\text{ALP}}_{k+1,\widetilde{\pi}})$, i.e., Algorithm 2 to get $J^{\text{ALP}}_{k,\widetilde{\pi}}$.
    $\pi = \widetilde{\pi}$
  until $J^{\text{ALP}}_{k,\widetilde{\pi}}(x) \geq J^{\text{ALP}}_{k,\pi}(x),\ \forall x \in S$
  Return optimal policy function for the kth stage.
end for

III. Proposed Algorithms and Theoretical Analysis

This section describes our proposed methodology, i.e., approximate linear programming (ALP) for decentralized policy iteration (DPI) in cooperative multi-agent MDPs (CO-MA-MDPs). In this work, we

consider finite horizon as well as infinite horizon CO-MA-MDPs. The proposed algorithms are written in a modular manner for ease of understanding.

A. DPI using ALP in Finite Horizon CO-MA-MDPs

This section demonstrates the proposed algorithm (Algorithm 3) for finite horizon CO-MA-MDPs. In Algorithm 3, ALP is implemented in decentralized policy iteration, where approximate policy evaluation is introduced unlike in [11].

The inputs in Algorithm 3 are the finite horizon length N, the number of states n and the number of agents m. Further, the state space S, action space $\mathcal{U}$ and base policy π are also taken as inputs. We denote the action space at the kth stage as $\mathcal{U}_k = \mathcal{U}_k^1 \times \mathcal{U}_k^2 \times \cdots \times \mathcal{U}_k^m$, where the action component for the ith agent is $u_k^i \in \mathcal{U}_k^i$. At first, we call CACFN$(k, \pi, J_{k+1,\pi}^{\text{ALP}})$ from Algorithm 2 to get the cost function value (at the kth stage) $J_{k,\pi}^{\text{ALP}}$. This algorithm takes the current stage number k, current policy π and the next stage cost $J_{k+1,\pi}^{\text{ALP}}$ as input and returns the cost at the kth stage. In this algorithm, the approximate cost is calculated using a linear program (see Lines 2 and 3 in Algorithm 2). In other words, we get the approximate cost $J_{k,\pi}^{\text{ALP}}$ as follows:

$$\begin{aligned}
&J_{k,\pi}^{\text{ALP}} = \max_{r \in \mathbb{R}^d} c^\top \Phi r, \quad c \in \mathbb{R}^n, \Phi \in \mathbb{R}^{n \times d} \\
&\text{Subject to: } \Phi r(x) \leq \sum_{y \in S} p_{xy}({\mu_k}^1(x), \ldots, {\mu_k}^m(x)) \\
&[g_{xy}({\mu_k}^1(x), \ldots, {\mu_k}^m(x)) + J_{k+1,\pi}^{\text{ALP}}(y)], \forall x \in S.
\end{aligned} \tag{6}$$

In this finite horizon MA-MDP algorithm, the terminal cost according to the base policy π, i.e., $J_{N,\pi}^{\text{ALP}}$ and the updated policy $\tilde{\pi}$, i.e., $J_{N,\tilde{\pi}}^{\text{ALP}}$ is the single stage cost g_N. As this is a finite horizon MDP, we perform decentralized policy iteration (DPI) using dynamic programming going backward in time, i.e., we first calculate the optimal policy for the $(N-1)$-stage problem using the fact that the optimal single-stage terminal cost is g_N. Next, for the $(N-2)$-stage problem, we use the optimal cost obtained for the $(N-1)$-stage problem and the process is repeated. At every stage, we repeat the (i) Decentralized policy improvement, and (ii) Approximate policy evaluation step (which is unlike [11], [12]) until the policy converges, i.e., until the cost, according to the updated policy, $J_{k,\tilde{\pi}}^{\text{ALP}}$ is greater than or equals the cost obtained according to the base policy, $J_{k,\pi}^{\text{ALP}}$. The procedure then returns the optimal policy for the kth stage.

In the Decentralized policy improvement step, each agent i updates its own policy component assuming that other agents are working as per their individual optimal policies. Now, $\tilde{\mu}_{1:i-1,k} = (\tilde{\mu}_k^1, \ldots, \tilde{\mu}_k^{i-1})$ denotes collectively the updated policies of agents $1, \ldots, i-1$ and $\mu_{i+1:m,k} = (\mu_k^{i+1}, \ldots, \mu_k^m)$ denotes collectively the *base* policy of agents $i+1, \ldots, m$ at the kth stage. In this step, for each agent i, Algorithm 1 is called, providing agent i, the current policy of other agents as mentioned above, and also the next stage cost $J_{k+1,\pi}^{\text{ALP}}$ as input, to obtain the updated policy component $\widetilde{\mu}_k^i$ of the ith agent at the kth stage. In other words, we get the updated policy component of the ith agent in state x as follows:

$$\begin{aligned}
&\widetilde{\mu}^i(x) \in \arg\min_{u^i \in \mathcal{U}^i(x)} \sum_{y \in S} p_{xy}(\widetilde{\mu}^1(x), \ldots, \widetilde{\mu}^{i-1}(x), u^i, \ldots, \\
&\mu^m(x))[g_{xy}(\widetilde{\mu}^1(x), \ldots, u^i, \ldots, \mu^m(x)) + \alpha J_{k+1,\pi}^{\text{ALP}}(y)]
\end{aligned} \tag{7}$$

Next, the updated policy $\tilde{\pi}$ is evaluated by calling CACFN$(k, \widetilde{\pi}, J_{k+1,\widetilde{\pi}}^{\text{ALP}})$, i.e., Algorithm 2, where current stage k, updated policy $\tilde{\pi}$ and next stage cost $J_{k+1,\tilde{\pi}}^{\text{ALP}}$ are provided as input. Next, the updated policy $\tilde{\pi}$ is set as base policy π. After the completion of execution of Algorithm 3, we receive the optimal policy and the corresponding optimal cost for all stages. Now, we introduce Theorem 1 to show a cost improvement property for Algorithm 3.

Let π be the base policy and $\widetilde{\pi}$ be the corresponding roll-out policy obtained from Algorithm 3. Let,

$$J_{k+1,\widetilde{\pi}}(x) \leq J_{k+1,\pi}(x) + \beta_{k+2}, \ \forall \ x \in S \ and \ k, \tag{8}$$

with $\beta_{k+2} = \max_{x \in S} |Err_{k+2}(x)|$ as the largest error $Err_{k+2}(x) := J_{k+1,\pi}(x) - J^{\text{ALP}}_{k+1,\pi}(x)$, $x \in S$. Let $\beta := \max\{\beta_N, \beta_{N-1} \ldots \beta_{k+1}, \ldots \beta_1\}$. Rewriting (8), we obtain for all $x \in S$, $k = 0, 1, \ldots, N-1$

$$\begin{aligned} J_{k+1,\widetilde{\pi}}(x) &\leq J_{k+1,\pi}(x) + (N-k-1)\beta, & (9) \\ J_{N,\widetilde{\pi}}(x) &= J_{N,\pi}(x) = g_N(x). & (10) \end{aligned}$$

Theorem 1: The following inequality holds:

$$J_{k,\widetilde{\pi}}(x) \leq J_{k,\pi}(x) + (N-k)\beta, \ \forall \ x \in S \text{ and } k. \tag{11}$$

Proof: For simplicity, we prove Theorem 1 for the two-agent case, i.e., $m = 2$. The proof for an arbitrary number of agents $m > 2$ is entirely similar. We use mathematical induction for our proof. Clearly, (11) holds for stage $j = N$, as $J_{N,\widetilde{\pi}} = J_{N,\pi} = g_N$. Thus, assuming that (11) holds for stage $j = k+1$, i.e., $J_{k+1,\widetilde{\pi}}(x) \leq J_{k+1,\pi}(x) + (N-k-1)\beta$, $\forall x$, we show that it also holds for $j = k$, thereby completing the induction step. Thus, note that

$$\begin{aligned}
&J_{k,\widetilde{\pi}}(x) \\
&= \sum_{y \in S} p_{xy}(\widetilde{\mu}^1_k(x), \widetilde{\mu}^2_k(x))[g_{xy}(\widetilde{\mu}^1_k(x), \widetilde{\mu}^2_k(x)) + J_{k+1,\widetilde{\pi}}(y)] \\
&\overset{(a)}{\leq} \sum_{y \in S} p_{xy}(\widetilde{\mu}^1_k(x), \widetilde{\mu}^2_k(x))[g_{xy}(\widetilde{\mu}^1_k(x), \widetilde{\mu}^2_k(x)) + J_{k+1,\pi}(y)] + (N-k-1)\beta \\
&= \sum_{y \in S} p_{xy}(\widetilde{\mu}^1_k(x), \widetilde{\mu}^2_k(x))[g_{xy}(\widetilde{\mu}^1_k(x), \widetilde{\mu}^2_k(x)) + (J^{\text{ALP}}_{k+1,\pi}(y) + Err_{k+2}(y))] + (N-k-1)\beta \\
&\leq \sum_{y \in S} p_{xy}(\widetilde{\mu}^1_k(x), \widetilde{\mu}^2_k(x))[g_{xy}(\widetilde{\mu}^1_k(x), \widetilde{\mu}^2_k(x)) + J^{\text{ALP}}_{k+1,\pi}(y)] + (N-k)\beta \\
&\overset{(b)}{=} \min_{u^2_k \in \mathcal{U}^2_k(x)} \sum_{y \in S} p_{xy}(\widetilde{\mu}^1_k(x), u^2_k)[g_{xy}(\widetilde{\mu}^1_k(x), u^2_k) + J^{\text{ALP}}_{k+1,\pi}(y)] + (N-k)\beta
\end{aligned}$$

Following similar steps for the 1st agent as for the 2nd agent,

$$\begin{aligned}
J_{k,\widetilde{\pi}}(x) &\leq \sum_{y \in S} p_{xy}(\mu^1_k(x), \mu^2_k(x))[g_{xy}(\mu^1_k(x), \mu^2_k(x)) + J^{\text{ALP}}_{k+1,\pi}(y)] + (N-k)\beta \\
&\overset{(c)}{\leq} \sum_{y \in S} p_{xy}(\mu^1_k(x), \mu^2_k(x))[g_{xy}(\mu^1_k(x), \mu^2_k(x)) + J_{k+1,\pi}(y)] + (N-k)\beta = J_{k,\pi}(x) + (N-k)\beta
\end{aligned}$$

In the above, (a) follows from the induction hypothesis, (b) follows from (7) and (c) follows from the property of ALP solutions that $J^{\text{ALP}}_{k,\pi}(x) \leq J_{k,\pi}(x), \forall x \in S$. ■

Remark 1: Theorem 1 states that the policy achieved at each stage in Algorithm 3 is close to the policy obtained when exact value function is computed [11] up to an error bound $(N-k)\beta$. The error here depends on β and the time gap between stage k and the end of horizon. From Theorem 1, we can say that near the end of horizon, the error goes to zero, and in other stages, the error depends on the function approximator. If the function approximator gives precise representations, then the error goes to zero, and we get an approximate policy that is as good as the exact policy.

Algorithm 4 Calculate approximate cost function: CACFN(μ)

1: Input: Policy μ
2: $r_\mu = \arg\max_{r \in \mathbb{R}^d} c^\top \Phi r$, Subject to: $\mathcal{T}_\mu \Phi r \geq \Phi r$
3: Return $J_\mu^{\text{ALP}} = \Phi r_\mu$

Algorithm 5 DPI using ALP for IH CO-MA-MDP

Input: Base policy $\mu_0 = (\mu_0^1, \ldots, \mu_0^m)$; discount factor α.
$J_{\mu_0}^{\text{ALP}} = \text{CACFN}(\mu_0)$ (call Algorithm 4)
for $t = 0, \ldots, T-1$ **do**
  for agent $i = 1, 2, \ldots, m$ **do**
    $\widetilde{\mu}^i = \text{DPIm}(i, \tilde{\mu}_{1:i-1}, \mu_{i+1:m}, J_{\mu_t}^{\text{ALP}}, \alpha)$ (call Algo.1)
  end for
  $\mu_{t+1} = \widetilde{\mu}$.
  $J^{\text{ALP}} = \text{CACFN}(\mu_{t+1})$, (call Algorithm 4).
end for
Return policy μ_T.

B. DPI using ALP in Infinite Horizon CO-MA-MDPs

In Algorithm 5, ALP is implemented in decentralized policy iteration (DPI) for the infinite horizon CO-MA-MDPs. In this algorithm, approximate policy evaluation is performed, unlike in [11], [12], where exact policy evaluation is done in a regular policy iteration procedure. In Algorithm 5, the inputs are the state space S (with n states), action space $\mathcal{U}$, number of agents (m) and base policy $\mu = \{\mu^1, \mu^2, \ldots, \mu^m\}$. Here $\mathcal{U} = \mathcal{U}^1 \times \mathcal{U}^2 \times \ldots \mathcal{U}^{i-1} \times \mathcal{U}^i \times \mathcal{U}^{i+1}, \cdots \times \mathcal{U}^m$, where the action component for the ith agent in state $x \in S$ is $u^i \in \mathcal{U}^i(x)$. As this algorithm is designed for infinite horizon CO-MA-MDPs, we use a discount factor $\alpha \in (0, 1)$. At first, call Algorithm 4 to get initial cost J_μ^{ALP} according to the base policy μ. Then, we repeat the $i)$ decentralized policy improvement (DPIm) and $ii)$ approximate policy evaluation until we get the optimal policy, i.e., until the cost according to the updated policy, $J_{\tilde{\mu}}^{\text{ALP}}$ is at least the cost according to the base policy, J_μ^{ALP} for all states $x \in S$. In the DPIm step, for each agent i, Algorithm 1 is called to get the updated policy $\tilde{\mu}^i$ for the ith agent providing agent i, current policy μ and cost J_μ^{ALP} as input. The policy update is thus carried out according to (7). In the approximate policy evaluation step, Algorithm 4 is called to get the cost $J_{\tilde{\mu}}^{\text{ALP}}$ according to the updated policy $\tilde{\mu}$. In Theorem 2, we show a cost improvement property of the updated policy $\tilde{\mu}$ when using Algorithm 5.

Theorem 2: In Algorithm 5, let $\beta_t := \max_{x \in S} |J_{\mu_t}(x) - J_{\mu_t}^{\text{ALP}}(x)|$. We have
$J_{\mu_{t+1}}(x) \leq J_{\mu_t}(x) + \dfrac{1}{1-\alpha}\beta_t, \ \forall x \in S.$

Proof: For the sake of notational simplicity, we denote $\mu \triangleq \mu_t$, $\tilde{\mu} \triangleq \mu_{t+1}$ and $\beta \triangleq \beta_t$, respectively. For ease of exposition, we consider the case of two agents (i.e., $m = 2$), as the case of an arbitrary number m of agents with $m > 2$ is entirely analogous. For the case of $m = 2, \mu = \tilde{\mu}, J = J_\mu$, observe that for any $x \in S$,

$$
\begin{aligned}
&\mathcal{T}_{\widetilde{\mu}} J_\mu(x) \\
&= \sum_{y=1}^{n} p_{xy}(\widetilde{\mu}_1(x), \widetilde{\mu}_2(x))(g_{xy}(\widetilde{\mu}_1(x), \widetilde{\mu}_2(x)) + \alpha J_\mu(y)) \\
&\overset{(a)}{=} \sum_{y=1}^{n} p_{xy}(\widetilde{\mu}^1(x), \widetilde{\mu}^2(x))(g_{xy}(\widetilde{\mu}^1(x), \widetilde{\mu}^2(x)) + \alpha(J_\mu^{\text{ALP}}(y) + \beta)) \\
&\overset{(b)}{=} \min_{u^2 \in \mathcal{U}^2(x)} \sum_{y=1}^{n} p_{xy}(\widetilde{\mu}^1(x), u^2)(g_{xy}(\widetilde{\mu}^1(x), u^2) + \alpha J_\mu^{\text{ALP}}(y)) + \alpha\beta \\
&\leq \sum_{y=1}^{n} p_{xy}(\widetilde{\mu}^1(x), \mu^2(x))(g_{xy}(\widetilde{\mu}^1(x), \mu^2(x)) + \alpha J_\mu^{\text{ALP}}(y)) + \alpha\beta.
\end{aligned}
$$

Following similar steps for the 1st agent as the 2nd agent,

$$
\begin{aligned}
&\mathcal{T}_{\widetilde{\mu}} J_\mu(x) \leq \sum_{y=1}^{n} p_{xy}(\mu^1(x), \mu^2(x))(g_{xy}(\mu^1(x), \mu^2(x)) + \alpha J_\mu^{\text{ALP}}(y)) + \alpha\beta \\
&\overset{(c)}{\leq} \sum_{y=1}^{n} p_{xy}(\mu(x))(g_{xy}(\mu(x)) + \alpha J_\mu(y)) + \alpha\beta \\
&= J_\mu(x) + \alpha\beta.
\end{aligned}
$$

In the above, (a) follows from $J_\mu = J_\mu^{\text{ALP}} + \beta$, (b) follows from (7) and (c) follows from the property of the ALP solution that $J_\mu^{\text{ALP}}(x) \leq J_\mu(x), \forall x \in S$. Hence, $\forall x \in S$, $e = (1, 1, \ldots, 1)^\top$ being the vector of all elements being 1,

$$
\begin{aligned}
&\mathcal{T}_{\widetilde{\mu}} J_\mu(x) \leq J_\mu(x) + \alpha\beta \\
&\Rightarrow \mathcal{T}_{\widetilde{\mu}}^2 J_\mu(x) \leq \mathcal{T}_{\widetilde{\mu}}(J_\mu + \alpha\beta e)(x) \\
&\overset{(d)}{\Rightarrow} \mathcal{T}_{\widetilde{\mu}}^2 J_\mu(x) \leq \mathcal{T}_{\widetilde{\mu}} J_\mu(x) + \alpha^2\beta \\
&\Rightarrow \mathcal{T}_{\widetilde{\mu}}^2 J_\mu(x) \leq J_\mu(x) + \alpha\beta + \alpha^2\beta.
\end{aligned}
$$

In the above, (d) follows from (see RHS above (d))

$$
\begin{aligned}
\mathcal{T}_{\widetilde{\mu}}(J_\mu + \alpha\beta e)(x) = \textstyle\sum_{y=1}^{n} p_{xy}((\widetilde{\mu}_1(x), \widetilde{\mu}_2(x)) \\
(g_{xy}(\widetilde{\mu}_1(x), \widetilde{\mu}_2(x)) + \alpha(J_\mu(y) + \alpha\beta)) \\
= \mathcal{T}_{\widetilde{\mu}} J_\mu(x) + \alpha^2\beta.
\end{aligned}
$$

Iterating the above step an infinite number of times, we obtain that $J_{\widetilde{\mu}}(x) \leq J_\mu(x) + \frac{1}{1-\alpha}\beta$, $\forall x \in S$. ■

Remark 2: Theorem 2 demonstrates that the policy obtained using function approximation in Algorithm 5, is close to the policy obtained using exact value function in [11] up to an error bound of $\frac{1}{1-\alpha}\beta$.

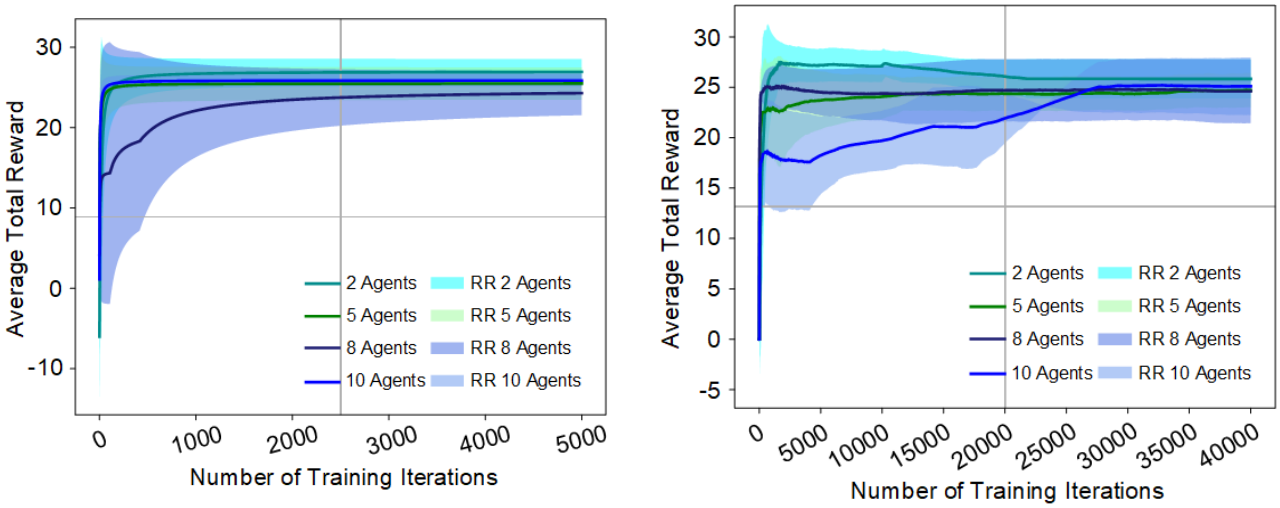
IV. Experiments and Results

In this section, we demonstrate the performance of the proposed algorithms, i.e., Algorithm 3 and Algorithm 5 on standard cooperative multi-agent MDP We implement both algorithms in the cost minimization as well as reward maximization applications (both cases are equivalent). For comparison with Algorithm 3 / Algorithm 5, we also implement the "Dynamic Programming (DP) / Policy Iteration (PI) for FH/IH CO-MA-MDP" using exact value function [11], and "Fully Decentralized DP/PI for FH/IH CO-MA-MDP" with function Approximation (Fully Dec. DP/PI) [13].

1. Cost minimization application: We consider the Flies-Spiders application [11], but in a two-dimensional (2-D) grid world (i.e., App. 1). In this 2-D grid world with 16 states, there are two spiders (agents) and two flies (goal states), where the position of the flies is fixed and the initial position of spiders is any random location in the grid. At each time instant, spiders can move one step in one of the four directions, i.e., "up", "down", "left", or "right". For the edge states in this grid world, an action that can take the agent out of the grid gets a penalty but doesn't change its location. In this application, the single-stage cost is $+1$, and penalty is $+2$ if spiders collide with each other, until both the flies are caught by the two spiders. In this application, in Algorithm 3, we implement two different horizon lengths of $N = 10$ and $N = 15$, respectively, and the goal is to cover all the goal states at a minimum cost. Further, for the infinite horizon discounted cost MA-MDP problem, we convert the problem into a continuing (non-episodic) task by randomly generating the initial positions of the spiders once they eat the flies and putting back the flies in the same positions as before. Here, we implement Algorithm 5 and our objective is to minimize the total expected discounted cost of the two agents. Table I presents the empirical results of our proposed algorithms and corresponding comparison results on the App. 1.

TABLE I: Comparison Results in Flies-Spiders application.

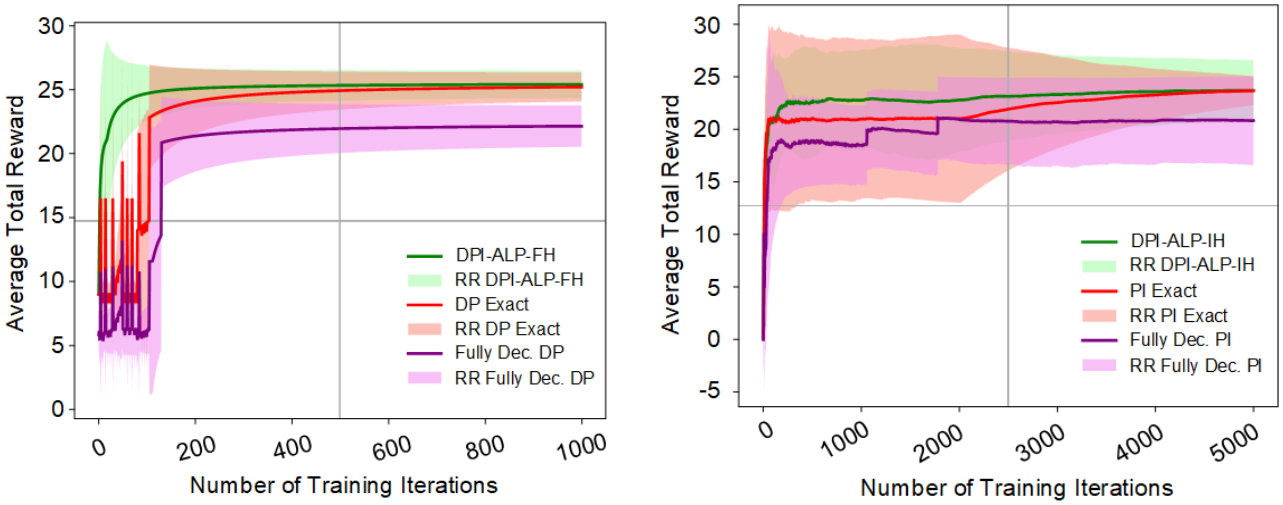
Algorithm	# Iterations to converge	Time (Sec.)
Dynamic Programming for FH CO-MA-MDP [11]	12	900.9
DPI using ALP for FH CO-MA-MDP (Proposed)	6	**100.4**
Regular Policy Iteration for IH CO-MA-MDP [11]	200	485.3
DPI using ALP for IH CO-MA-MDP (Proposed)	1	**25.6**



(a) (b)

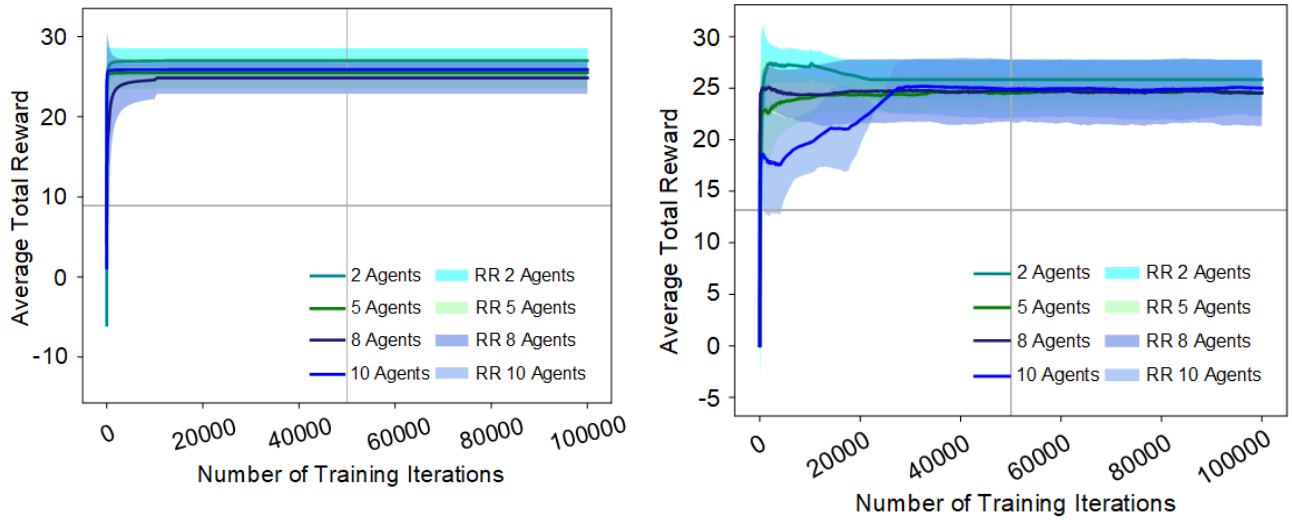
Fig. 1: Performance in terms of average reward and standard deviation obtained from (a) Algorithm 3, (b)Algorithm 5, for different numbers of agents on App. 2.

2. Reward maximization application: We further extended our experiments to grid world environment (App. 2) with 2500 states and 5 actions, i.e., go one step "up", "down", "left", "right" or "stay" at the



(a) (b)

Fig. 2: Performance comparisons ($m = 5$) of (a) Algorithm 3, (b) Algorithm 5 with other algorithms on App. 2.

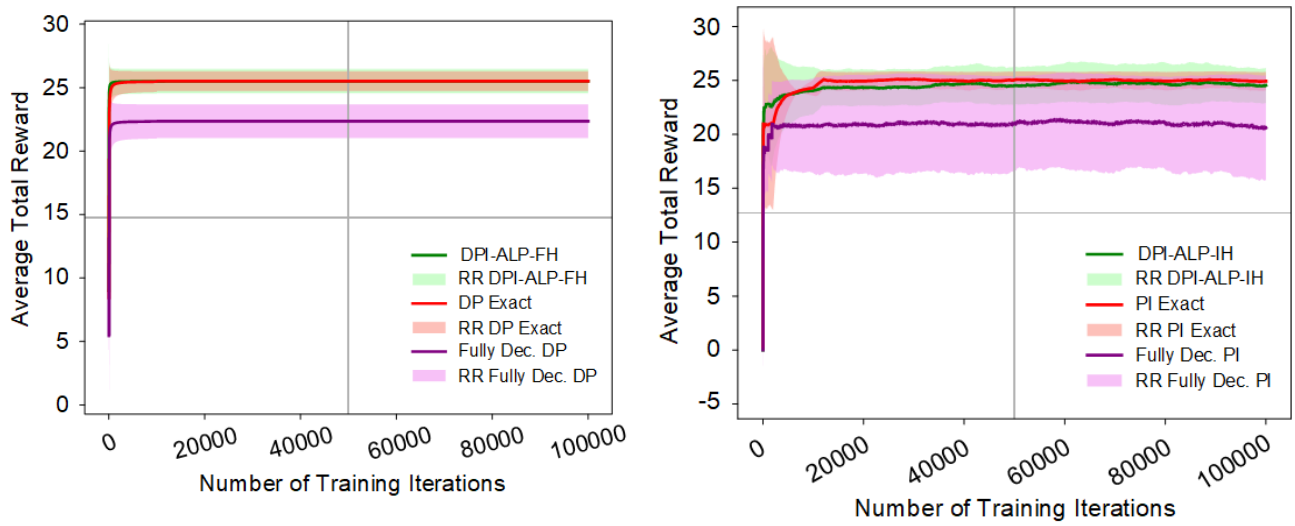


(a) (b)

Fig. 3: Performance in terms of average reward and standard deviation obtained from (a) Algorithm 3, (b) Algorithm 5 for different number of agents for 100,000 iterations on App. 2.

same location. In these experiments, our objective is to maximize the long-term collaborative reward for m agents with $m = 2, 5, 8, 10$. The implementation environment of Algorithm 3 has an equal number of goal states as the number of agents and when an agent reaches its goal state, it receives a positive reward, otherwise a negative reward. Here we set $N = 50$. On the other hand, Algorithm 5 has some randomly generated positive reward state, i.e., if any agent enters this state, it will get a positive reward, and the reward for all other states is 0.

We obtain ten independent runs of the algorithms using different initial seeds. The average reward (mean) and standard deviation (std.) of rewards, respectively, obtained at each time instant from the ten runs, are calculated and plotted as a dark-colored line and a light-colored reward region (RR), respectively.



(a) (b)

Fig. 4: Performance comparisons ($m = 5$) of (a) Algorithm 3, (b) Algorithm 5 with other algorithms for 100,000 iterations on App. 2.

TABLE II: Performance comparisons ($m = 5$) of our proposed algorithms, i.e., Algorithm 3 and Algorithm 5, with [11], [13], averaged across 10 independent runs on App. 2.

Algorithm	**Mean $\pm$ Std.**	**# Iterations to converge**
Dynamic Programming for FH CO-MA-MDP [11]	25.5 ± 0.78	1000
Fully Decentralized FH CO-MA-MDP	22.35 ± 1.32	150
DPI using ALP for FH CO-MA-MDP (Proposed)	**25.5 $\pm$ 0.95**	**100**
Regular Policy Iteration for IH CO-MA-MDP [11]	24.94 ± 0.86	4500
Fully Decentralized IH CO-MA-MDP [13]	20.62 ± 4.85	1800
DPI using ALP for IH CO-MA-MDP (Proposed)	**24.56 $\pm$ 1.62**	**200**

The performance of our algorithms during training and the comparison with existing algorithms are graphically presented in Figure 1 and Figure 2, respectively. We run $100,000$ iterations of our proposed algorithms and the algorithms used for comparison. However, we show these plots for a lower number of iterations for better visualization of the speed of convergence of each algorithm. Nonetheless, we also show, in Figure 3 and Figure 4, the plots over 100,000 runs of all algorithms. In each figure, the x-axis represents the number of training iterations, and the y-axis represents the average total reward obtained so far (calculated as explained previously). Further, in Table II, we summarize the final values of the mean and standard error of the average reward obtained from the ten independent runs after all the algorithms have been run for 100,000 iterations as well as the average number of iterations needed for each algorithm to converge. We show this as a representative table here for the case of $m = 5$. We make the following observations from the experiments using our algorithms on the aforementioned

applications (i.e., App. 1 and App. 2):

1. Parts (a)–(b) of Figure 1 show that the average total reward of Algorithm 3 and Algorithm 5, respectively, converges for all four considered values of m.

2. Tables I–II and parts (a)–(b) of Figure 2 demonstrate that our proposed algorithms converge faster than those of [11] and [13]. Table I shows that our proposed algorithm for the finite horizon setting is nearly 9 times faster than DP. Further, our algorithm for the infinite horizon setting is more than 19 times faster than regular PI.

3. Table II, parts (a)–(b) of Fig. 2 show that the proposed algorithms achieve equal or higher average total reward compared to exact value [11]/fully dec. CO-MA-MDP [13].

4. Because of function approximation in our algorithms, unlike [11], when 10 agents ($m = 10$), 2500 states ($\mid \mathcal{S} \mid = 2500$), and 30 basis functions ($d = 30$) are used, we get a dimensionality reduction factor $\frac{|\mathcal{S}|^m}{d} = 3.2 * 10^{32}$.

V. Conclusions

In this paper, we consider multi-agent Markov decision processes (MA-MDP) with cooperative agents under two different settings: (a) finite horizon and (b) infinite horizon, respectively. We separately present a novel algorithm for each of the two settings where we use approximate linear programming for decentralized policy iteration in CO-MA-MDP. We prove the convergence of both our proposed algorithms and show the applications of these to standard cooperative MA-MDP tasks. Our experimental results confirm that our algorithms outperform competing algorithms in the literature, involving significantly fewer iterations while providing better performance.

References

[1] D. Bertsekas, "Dynamic programming and optimal control," *Athena Scientific*, vol. I, 2017.

[2] M. Sayin and *et al.*, "Decentralized Q-learning in zero-sum Markov games," in *NeurIPS*, 2021, pp. 1–15.

[3] G. Ramponi and et al., "Learning in non-cooperative configurable Markov decision processes," *NeurIPS*, pp. 22 808–22 821, 2021.

[4] P. Corrêa and H. T. Vieira, "Cooperative Markov decision process for voltage control of grid-tied photovoltaic generators with reactive power support," *IEEE Trans. Sus. Ene.*, vol. 13, no. 2, pp. 919–933, 2021.

[5] Y. Rizk and *et al.*, "Decision making in multiagent systems: A survey," *IEEE Trans. on Cog. Dev. Sys.*, vol. 10, no. 3, pp. 514–529, 2018.

[6] R. S. Sutton and A. G. Barto, "Reinforcement learning : An introduction, 2'nd ed." *Cambridge, MA: MIT Press*, vol. 2, p. 526, 2018.

[7] M. Li and et al., "Multiplayer Stackelberg-Nash Game for Nonlinear System via Value Iteration-Based Integral Reinforcement Learning," *IEEE Tran. Neu. Net. Lea. Sys.*, vol. 33, no. 4, pp. 1429–1440, 2022.

[8] W. B. Powell, *Approximate Dynamic Programming: Solving the curses of dimensionality*. John Wiley & Sons, 2007, vol. 703.

[9] C. Wang and et al., "Mdp-based distribution network reconfiguration with renewable distributed generation: Approximate dynamic programming approach," *IEEE T Sm. Gr.*, vol. 11, no. 4, pp. 3620–3631, 2020.

[10] T. Campbell and *et al.*, "Multiagent allocation of Markov decision process tasks," in *2013 Ame. Cont. Conf.*, 2013, pp. 2356–2361.

[11] D. Bertsekas, "Multiagent Reinforcement Learning: Rollout and Policy Iteration," *IEEE/CAA J. of Auto. Sin.*, vol. 8, no. 2, pp. 249–272, 2021.

[12] ——, *Rollout, Policy Iteration, and Distributed Reinforcement Learning*. Athena Scientific, 2020.

[13] Z. Chen and *et al.*, "Sample and Communication-Efficient Decentralized Actor-Critic Algorithms with Finite-Time Analysis," in *ICML*, 2022.

[14] C. Lakshminarayanan, S. Bhatnagar, and C. Szepesvari, "A linearly relaxed approximate linear program for markov decision processes," *IEEE Tran. on Auto. cont.*, vol. 63, no. 4, pp. 1185–1191, 2017.